Machine Learning - Assignment 2

Multiclass classification of dry beans with five classifiers and a Streamlit evaluation app - M.Tech Artificial Intelligence & Machine Learning (AIML)

Field	Details
Name	Basetti Shanmugam
Student ID / Roll no.	2026AJ05044
Email	2026aj05044@wilp.bits-pilani.ac.in
Programme / Course	M.Tech AIML - Machine Learning
Submission date	08/11/2026

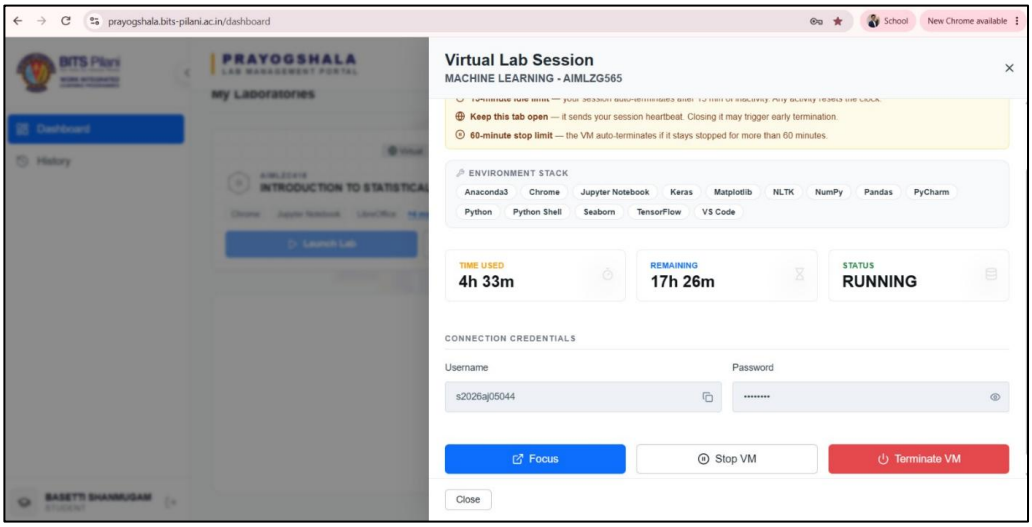
1. GitHub repository link

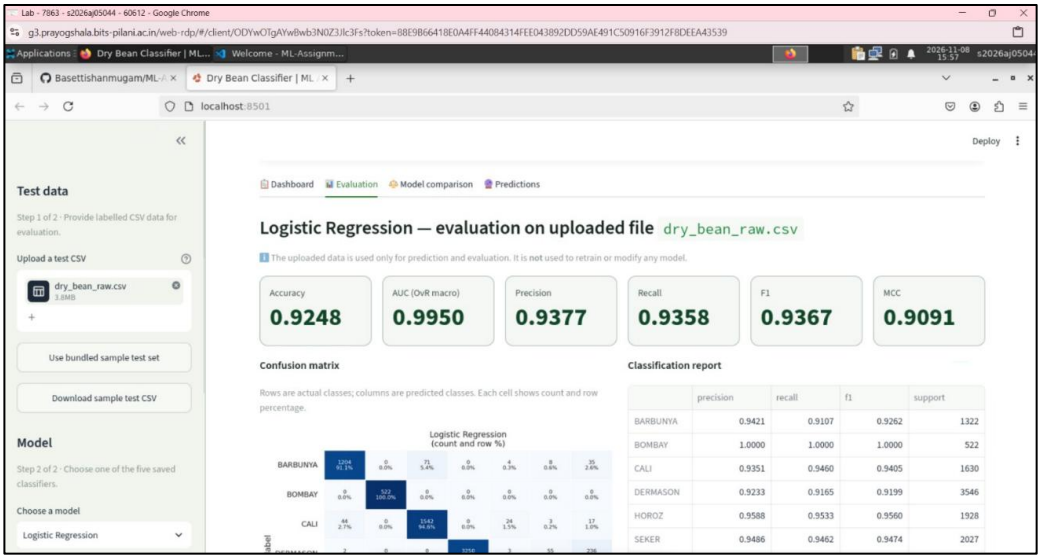
https://github.com/Basettishanmugam/ML-Assignment-2

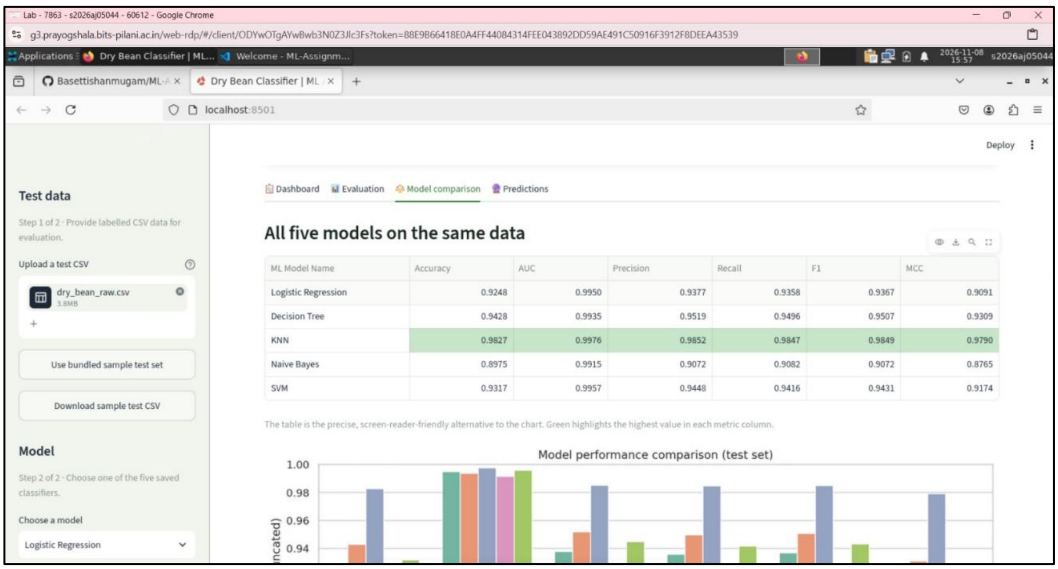
2. Live Streamlit application link

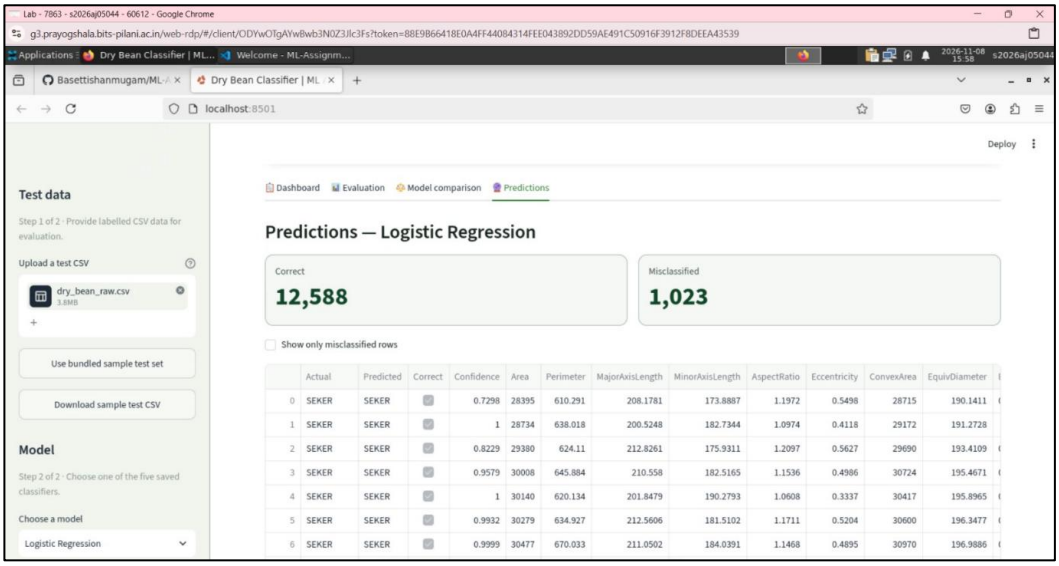
https://bits-2026aj05044-ml-assignment-2.streamlit.app/

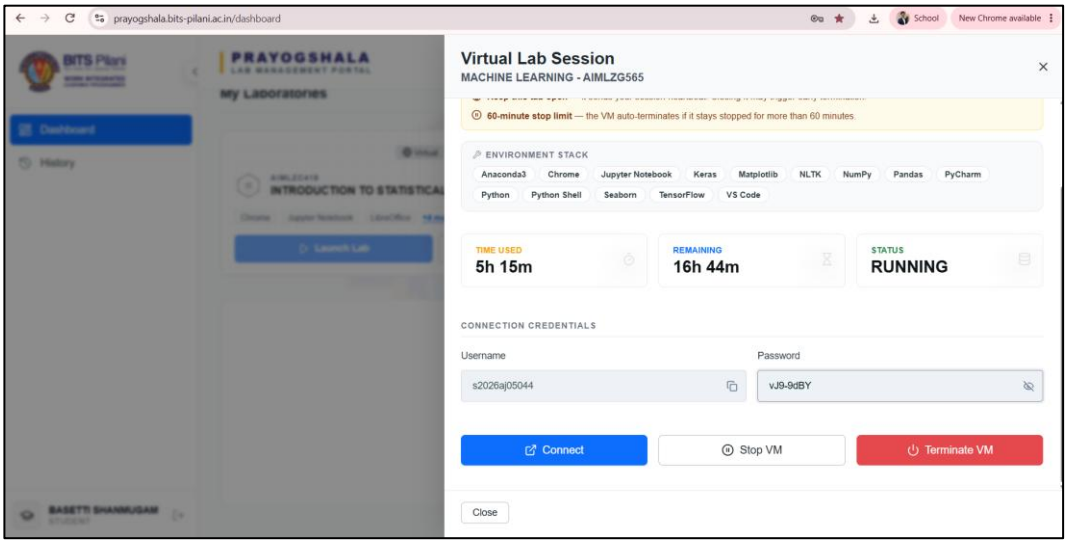
3. BITS Virtual Lab / Prayogshala screenshot






Test data
Step 1 of 2 · Provide labelled CSV data for evaluation.
Upload a test CSV
dry_bean_raw.csv
Use bundled sample test set
Download sample test CSV
Model
Step 2 of 2 · Choose one of the five saved classifiers.
Choose a model
Logistic Regression
Deploy
Dashboard
Evaluation
Model comparison
Predictions
All five models on the same data
ML Model Name | Accuracy | AUC | Precision | Recall | F1 | MCC
Logistic Regression | 0.9248 | 0.9950 | 0.9377 | 0.9358 | 0.9367 | 0.9091
Decision Tree | 0.9428 | 0.9935 | 0.9519 | 0.9496 | 0.9507 | 0.9309
KNN | 0.9827 | 0.9976 | 0.9852 | 0.9847 | 0.9849 | 0.9790
Naive Bayes | 0.8975 | 0.9915 | 0.9072 | 0.9082 | 0.9072 | 0.8765
SVM | 0.9317 | 0.9957 | 0.9448 | 0.9416 | 0.9431 | 0.9174
The table is the precise, screen-reader-friendly alternative to the chart. Green highlights the highest value in each metric column.
Model performance comparison (test set)
1.00
0.98
0.96
0.94


Test data
Step 1 of 2 · Provide labelled CSV data for evaluation.
Upload a test CSV
dry_bean_raw.csv
Use bundled sample test set
Download sample test CSV
Model
Step 2 of 2 · Choose one of the five saved classifiers.
Choose a model
Logistic Regression
Deploy
Dashboard
Evaluation
Model comparison
Predictions
Predictions — Logistic Regression
Correct
12,588
Misclassified
1,023
Show only misclassified rows
Actual | Predicted | Correct | Confidence | Area | Perimeter | MajorAxisLength | MinorAxisLength | AspectRatio | Eccentricity | ConvexArea | EquivDiameter
0 SEKER SEKER 0.7298 28395 610.291 208.1781 173.8887 1.1972 0.5498 28715 190.1411
1 SEKER SEKER 1 28734 638.018 200.5248 182.7344 1.0974 0.4118 29172 191.2728
2 SEKER SEKER 0.8229 29380 624.11 212.8261 175.9311 1.2097 0.5627 29690 193.4109
3 SEKER SEKER 0.9579 30008 645.884 210.558 182.5165 1.1536 0.4986 30724 195.4671
4 SEKER SEKER 1 30140 620.134 201.8479 190.2793 1.0608 0.3337 30417 195.8965
5 SEKER SEKER 0.9932 30279 634.927 212.5606 181.5102 1.1711 0.5204 30600 196.3477
6 SEKER SEKER 0.9999 30477 670.033 211.0502 184.0391 1.1468 0.4895 30970 196.9886


prayogshala.bits-pilani.ac.in/dashboard
PRAYOGSHALA
My Laboratories
Virtual Lab Session
MACHINE LEARNING - AIMLZG565
60-minute stop limit — the VM auto-terminates if it stays stopped for more than 60 minutes
ENVIRONMENT STACK
Anaconda3 Chrome Jupyter Notebook Keras Matplotlib NLTK NumPy Pandas PyCharm
Python Python Shell Seaborn TensorFlow VS Code
TIME USED
5h 15m
REMAINING
16h 44m
STATUS
RUNNING
CONNECTION CREDENTIALS
Username
s2026aj05044
Password
vJ9-9dBY
Connect
Stop VM
Terminate VM
Close

4. README / report content

Dry Bean Classification - Machine Learning Assignment 2

M.Tech Artificial Intelligence & Machine Learning (AIML)

a. Problem Statement

Dry beans of different registered varieties look alike, and manual sorting is slow and subjective. The task is to build, evaluate and compare five supervised classifiers that predict the bean variety (Class, 7 categories) from 16 geometric measurements obtained from grain images. All models use the same data and the same stratified 80:20 train/test split and are scored with Accuracy, AUC, Precision, Recall, F1 and MCC. The best model is then exposed through an interactive Streamlit application.

b. Dataset Description

- Name: Dry Bean Dataset (UCI ID 602) - multiclass classification
- Instances: 13,611 (raw) -> 13,543 after removing 68 exact duplicate rows
- Input features: 16 numeric (12 dimension features + 4 shape factors) - assignment minimum is 12
- Target: Class - 7 classes: BARBUNYA, BOMBAY, CALI, DERMASON, HOROZ, SEKER, SIRA
- Missing values: 0. Class counts after cleaning: DERMASON 3546, SIRA 2636, SEKER 2027, HOROZ 1860, CALI 1630, BARBUNYA 1322, BOMBAY 522 (imbalance ratio 6.8:1).

#	Feature	Description
1	Area	Number of pixels inside the bean region
2	Perimeter	Length of the bean border
3	MajorAxisLength	Longest line that can be drawn through the bean
4	MinorAxisLength	Longest line perpendicular to the major axis
5	AspectRatio	MajorAxisLength / MinorAxisLength
6	Eccentricity	Eccentricity of the ellipse
7	ConvexArea	Pixels in the smallest convex polygon containing the bean
8	EquivDiameter	Diameter of a circle with the same area
9	Extent	Bounding-box pixels / bean area
10	Solidity	Convex-hull pixels / bean pixels
11	Roundness	4pi A / P^2
12	Compactness	EquivDiameter / MajorAxisLength
13	ShapeFactor1	Shape factor 1
14	ShapeFactor2	Shape factor 2
15	ShapeFactor3	Shape factor 3
16	ShapeFactor4	Shape factor 4

c. Dataset Source and Attribution

- UCI Machine Learning Repository: https://archive.ics.uci.edu/dataset/602/dry+bean+dataset (DOI https://doi.org/10.24432/C50S4B)
- Kaggle mirror: https://www.kaggle.com/datasets/muratkokludataset/dry-bean-dataset
- Authors' dataset page: https://www.muratkoklu.com/datasets/
- License: Creative Commons Attribution 4.0 (CC BY 4.0), so redistribution with attribution is permitted; the Excel file is bundled in data/raw/.
- Citation: KOKLU, M. and OZKAN, I.A. (2020), Multiclass Classification of Dry Beans Using Computer Vision and Machine Learning Techniques. Computers and Electronics in Agriculture, 174, 105507. DOI: https://doi.org/10.1016/j.compag.2020.105507
- UCI citation: Dry Bean [Dataset]. (2020). UCI Machine Learning Repository. https://doi.org/10.24432/C50S4B

Two loaders are provided in the notebook (Option A UCI via ucimlrepo, Option B downloaded Kaggle/author file); both are converted to one standard schema by src/data_loader.py. Option B was used to produce all results in this repository.

d. GitHub Repository Link

https://github.com/Basettishanmugam/ML-Assignment-2

e. Data Preprocessing

1. Features and target separated; schema validated (16 features + Class).
2. Missing values: none found (the code would drop such rows; nothing is imputed or invented).
3. Duplicates: 68 exact duplicate rows removed (first copy kept) before splitting, so no sample can appear in both train and test.
4. Target encoded with LabelEncoder (saved in models/label_encoder.joblib).
5. Stratified 80:20 split, random_state=42 -> 10,834 train / 2,709 test, shared by all five models.
6. StandardScaler is the first step of every Pipeline, therefore fitted on training data only (no leakage).
7. Outliers are kept (genuine measurements); nothing in the data is modified.

f. Models Used

Model	Pipeline	Key hyper-parameters (fixed a priori, not tuned on test data)
Logistic Regression	StandardScaler -> LogisticRegression	max_iter=2000, L2, C=1
Decision Tree	StandardScaler -> DecisionTreeClassifier	max_depth=10, min_samples_leaf=5
K-Nearest Neighbors	StandardScaler -> KNeighborsClassifier	n_neighbors=7, weights=distance
Naive Bayes	StandardScaler -> GaussianNB	defaults
SVM	StandardScaler -> SVC	RBF, C=1, gamma=scale, probability=True

How each works, why it suits this data and its pros/cons are explained in the notebook (Phase 4).

g. Model Evaluation Metrics

Computed on the 2,709-row held-out test set; multiclass settings: Accuracy; AUC = one-vs-rest, macro-averaged, from predict_proba; Precision / Recall / F1 = macro-averaged; MCC = multiclass Matthews correlation coefficient. zero_division=0 handles classes that are never predicted.

h. Model Comparison Table

ML Model Name	Accuracy	AUC	Precision	Recall	F1	MCC
Logistic Regression	0.9192	0.9947	0.9307	0.9300	0.9302	0.9023
Decision Tree	0.9070	0.9779	0.9220	0.9204	0.9211	0.8875
KNN	0.9136	0.9854	0.9268	0.9237	0.9250	0.8956
Naive Bayes	0.8970	0.9911	0.9081	0.9082	0.9072	0.8762
SVM	0.9210	0.9945	0.9342	0.9317	0.9328	0.9045

Additional generalisation diagnostics (reports/model_comparison_details.csv; CV uses the training data only):

Model	Train acc.	5-fold CV acc.	Test acc.	Train-test gap
Logistic Regression	0.9260	0.9243 +- 0.0030	0.9192	0.0068
Decision Tree	0.9518	0.9112 +- 0.0059	0.9070	0.0448
KNN	1.0000	0.9240 +- 0.0055	0.9136	0.0864
Naive Bayes	0.8972	0.8968 +- 0.0060	0.8970	0.0002
SVM	0.9342	0.9300 +- 0.0057	0.9210	0.0132

i. Model Performance Observations

ML Model Name	Observation About Model Performance
Logistic Regression	Second-best on Accuracy (0.9192), Precision, Recall, F1 and MCC, and the highest AUC (0.9947). Train-test accuracy gap is only 0.0068, so it generalises very well.
Decision Tree	Accuracy 0.9070 and the lowest AUC (0.9779): a tree outputs piecewise-constant leaf probabilities, so its class ranking is coarser. It shows the largest genuine overfitting among the models.

KNN	Mid-table: Accuracy 0.9136, F1 0.9250, AUC 0.9854. Its training accuracy is 1.0000 only because distance weighting lets each training point vote for itself.
Naive Bayes	Weakest on Accuracy (0.8970), F1 (0.9072) and MCC (0.8762), yet third-highest on AUC (0.9911): the independence assumption is violated by strong feature correlations.
SVM	Best on Accuracy (0.9210), Precision (0.9342), Recall (0.9317), F1 (0.9328) and MCC (0.9045); AUC (0.9945) is second, just 0.0002 behind Logistic Regression.

- Best model overall by accuracy: SVM - Highest AUC: Logistic Regression
- Best precision/recall balance: SVM (highest F1 = 0.9328; Precision 0.9342 and Recall 0.9317 are both the highest).
- Generalisation: Naive Bayes has the smallest gap (0.0002) but at lower accuracy; among the strong models Logistic Regression (0.0068) and SVM (0.0132) generalise best.
- Poor performers: Naive Bayes (lowest Accuracy/F1/MCC); Decision Tree has the lowest AUC and the biggest real overfitting gap.
- Per-class (SVM): hardest class is SIRA (F1 0.8684); easiest is BOMBAY (F1 1.0000).

j. Overall Winning Model

Overall winner: Support Vector Machine (RBF). It ranks first on five of the six metrics (Accuracy, Precision, Recall, F1, MCC) and second on AUC (only 0.0002 behind Logistic Regression), and has the best cross-validated accuracy on the training data. Its accuracy lead over Logistic Regression is 0.0018, smaller than one standard error of the test accuracy, so SVM and Logistic Regression should be regarded as statistically very close; Logistic Regression is the simpler, faster alternative.

k. Streamlit Application Description

app.py is a four-section dashboard suite with a sidebar menu (Dashboard - Model Performance - Prediction - About & Guide). It loads the five saved pipelines and never retrains. Every number shown is computed from the saved models.

- Dashboard: KPI cards, a bar chart for five models, class-distribution pie, and dataset facts (16 features, 7 classes, 13,611 records).
- Model Performance: comparison, metrics table, ROC curves, confusion matrices, and CSV download.
- Prediction: upload a CSV (validated for missing/unexpected columns, non-numeric or missing values, empty file, unknown class labels), pick a model, click Run Prediction, and view metrics, predictions, confusion matrix and report.
- About & Guide: project overview, dataset information, references and usage guidelines.

Saved-model / no-retraining deployment method. Training happens once, offline. Each model is saved as a complete scikit-learn Pipeline with joblib.dump, so scaler statistics travel with the model. At run time app.py only calls joblib.load, predict, predict_proba and metric functions; it contains no fit() call. Uploaded rows are used only for prediction and scoring.

l. Deployment Link

https://bits-2026aj05044-ml-assignment-2.streamlit.app/

m. Project Folder Structure

ML_Assignment_2/
|-- README.md
|-- PROJECT_OVERVIEW.md
|-- SUBMISSION_CHECKLIST.md
|-- SETUP_BY_OS.md
|-- requirements.txt
|-- requirements-dev.txt
|-- .gitignore
|-- app.py
|-- config.py
|-- data/ (README.md, raw/, processed/, test_data.csv)
|-- notebooks/ML_Assignment_2.ipynb
|-- src/ (data_loader.py, preprocessing.py, train_models.py, evaluate_models.py, visualization.py)
|-- models/ (five .joblib pipelines, label_encoder.joblib, metadata.json)
|-- reports/ (comparison CSVs, classification reports, figures)
|-- screenshots/README.md
|-- deployment/
|-- tests/run_checks.py
|-- tools/

n. Installation Instructions

Requires Python 3.11 or newer. Get the code, then create a virtual environment and install the pinned dependencies.

OS / shell	Create + activate the environment	Install
Windows PowerShell	py -3.12 -m venv .venv then .venv\Scripts\Activate.ps1	pip install -r requirements.txt
Windows Command Prompt	py -3.12 -m venv .venv then .venv\Scripts\activate.bat	pip install -r requirements.txt
Windows Git Bash	py -3.12 -m venv .venv then source .venv/Scripts/activate	pip install -r requirements.txt
macOS	python3 -m venv .venv then source .venv/bin/activate	pip install -r requirements.txt
Linux	python3 -m venv .venv then source .venv/bin/activate	pip install -r requirements.txt

If PowerShell blocks activation, run Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned once. Add pip install -r requirements-dev.txt if you want to run the notebook. scikit-learn is pinned to 1.8.0 because the .joblib files were saved with that version.

o. Execution Instructions

After activating .venv, the commands below are identical on Windows, macOS and Linux:

Purpose	Command
Train and save the five pipelines	python -m src.train_models
Metrics, reports, confusion matrices, figures	python -m src.evaluate_models
EDA figures	python -m src.visualization
Automated checks	python tests/run_checks.py
Launch the Streamlit app	streamlit run app.py
Same if streamlit is not recognised	python -m streamlit run app.py
Open the notebook	jupyter notebook notebooks/ML_Assignment_2.ipynb

To use Option A (UCI) in the notebook, un-comment data_loader.load_from_uci() and comment out the Option B lines in the loading cell.

p. Conclusion

All five classifiers reach roughly 90%-92% test accuracy on the 7-class Dry Bean problem, with macro-AUC between 0.978 and 0.995. SVM (RBF kernel) gives the best overall result, with Logistic Regression statistically very close and much simpler; Naive Bayes is weakest, consistent with the strong feature correlations that violate its independence assumption. The hardest distinction is DERMASON vs SIRA. Limitations: hyper-parameters were not tuned, a single train/test split was used for the headline numbers, and differences between the top two models are within sampling noise. Possible extensions include grid/random search with nested cross-validation, feature selection to remove redundant size features, and ensemble methods.

Note: the results in this README are generated from the CSV files in reports/; reports/TESTING_REPORT.md lists what was and was not tested.